

Lab 1: Asynchronous Messaging with AI Tool Integration

Week 4 | Labs Portfolio: 10% of the module, pass/fail | Duration: Two 2-hour sessions | Assessed in the session, in pairs

Learning Outcomes

- Implement a message broker (RabbitMQ or Kafka) for asynchronous communication
- Design a producer-consumer pattern for decoupled service communication
- Integrate a REST-based AI API as an asynchronous consumer
- Reflect on AI tool usage in a structured way using structured logs and architectural decision records

Prerequisites

- Java (Spring Boot) or Python (FastAPI) development environment
- Docker and Docker Compose installed
- Basic familiarity with REST APIs and JSON message formats
- Module text Ch. 5 — *Asynchronous Architecture and Event-Driven Systems* (assigned reading for Week 4)

Background

In Week 4, we explored the broker topology and event-driven architectures (EDA). This lab puts those concepts into practice. Asynchronous messaging enables decoupled communication: a producer sends a message to a queue and continues without waiting for a consumer to process it. This pattern is essential when dealing with long-running tasks such as machine learning model inference, data processing pipelines, or external API calls that may have high latency or fail temporarily. Real-world distributed systems—especially those involving AI—rely heavily on message brokers (RabbitMQ, Kafka, AWS SQS) to handle variable load and provide resilience.

In this lab, you will build a simple pipeline where a producer accepts requests, enqueues them, and a consumer processes them by calling an AI API (such as OpenAI's GPT, Anthropic's Claude, or a local LLM). You will experience firsthand why resilience patterns (dead letter queues, retries) matter when external dependencies—especially AI APIs with rate limits and non-deterministic latency—are involved.

Setup

Step 1: Spin up RabbitMQ

```
docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:4-management
```

This starts RabbitMQ with management UI enabled. Verify at <http://localhost:15672> (default login: guest/guest).

Step 2: Create project scaffold

For Java, create a Spring Boot project with Spring AMQP (via Spring Initializr or maven). For Python, install pika: `pip install pika fastapi uvicorn`.

Step 3: AI API access

Option A (Local, recommended): Install Ollama and pull a small model (e.g. `ollama pull llama3.2:1b`) — free, no rate limits, no account. **Option B (Cloud):** OpenAI or Anthropic API — both require paid credits; only choose this if you want to observe real production rate limits (429s), which is instructive for Task B.

Part 1: Guided Exercises (UNASSESSED — ~60 min)

Exercise 1.1: Write a simple producer

- Using Spring AMQP (Java) or pika (Python), write a service that sends messages to a RabbitMQ queue named `task_queue`.
- Each message should have the structure: `{"id": 1, "text": "Process this request", "timestamp": "2026-10-05T12:00:00Z"}`
- Send at least 10 messages and observe the queue depth in the RabbitMQ management UI (<http://localhost:15672>).

Exercise 1.2: Write a simple consumer

- Write a consumer service that listens on the same `task_queue`.
- For each message received, log: message ID, text, and current processing timestamp.
- Let the consumer process messages at a controlled rate (e.g., 1 per second) to observe queue depth drop.
- Run producer and consumer simultaneously—verify messages are consumed.

Exercise 1.3: Explore message acknowledgement and durability

- Test auto-ack vs. manual acknowledgement: modify the consumer to NOT acknowledge (`auto_ack=False` in Python, or manual mode in Java) and intentionally crash mid-processing. Restart and verify the message is reprocessed.
- Enable message persistence: set `durable=True` on the queue and `persistent=True` on messages. Stop RabbitMQ, restart it, and verify messages survive the broker restart.
- Document your observations in your lab notebook.

Part 2: Assessed Work (ASSESSED — ~90 min)

Task A: Async AI Processing Service

Fixed layout for the automated test harness: the Producer Service runs on port 8080 and exposes exactly `POST /process` and `GET /result/{id}`. Keep these paths and port — marking depends on them.

Build an end-to-end asynchronous service with the following components:

- **Producer Service:** Exposes a REST endpoint `POST /process` that accepts `{"text": "..."}` . Returns a request ID immediately (do not wait for processing). Enqueues the message to RabbitMQ.
- **Message Queue:** RabbitMQ queue with `durable=True` and persistent message delivery.
- **Consumer Service:** Reads messages from the queue. For each message, calls an AI API (OpenAI, Anthropic, or Ollama) with the text and stores the result (in-memory map or simple JSON file) mapped by request ID.
- **Results Endpoint:** `GET /result/{id}` retrieves the stored AI response or a status message (`"processing"`, `"completed"`, `"error"`).
- **Graceful Failure:** If the AI API is unavailable (timeout, 503, rate limit), log the failure and requeue the message (do not lose it). Set a max retry count to avoid infinite loops.

Task B: Error Handling and Dead Letter Queue (DLQ)

Implement a Dead Letter Queue to handle permanently failed messages:

- Configure a DLQ exchange and binding: after N failed retries (e.g., 3), route the message to a dead letter queue.
- Test the DLQ by pointing the AI API endpoint to a non-existent URL and observe failed messages accumulate in the DLQ.

- Verify using the RabbitMQ management UI. Document the DLQ configuration (queue declaration, bindings, TTL settings if used).
- Provide a brief explanation of when and why DLQs are essential in production systems.

Task C: Documentation

- Write one Architectural Decision Record (ADR): Document your choice of RabbitMQ vs. Kafka for this use case. Include Context, Decision, Consequences, and Alternatives Considered.
- Create an architecture diagram (hand-drawn or using draw.io) showing: producer, queue, consumer, AI API, results store, and DLQ. Include arrows for message flow.

AI Tool Integration

You are encouraged to use GitHub Copilot, ChatGPT, Claude, or Gemini while completing this lab. For example, you might ask an AI tool to generate starter code for a Spring Boot producer, or help debug a pika consumer. However, you must reflect critically on the AI's output: Did it give you correct code? Did you need to modify it? Did it miss error handling?

Important: Do NOT blindly copy AI-generated code. Understand what you submit — Lab 1 material is in scope for the supervised lab exam in Week 10 (no AI assistants there), and your project viva may probe any pattern used here.

Appendix A: Architectural Decision Record (ADR) Template

```
ARCHITECTURAL DECISION RECORD (ADR)
-----
Title: [Short descriptive title]
Date: [Date]
Status: [Proposed / Accepted / Deprecated]
Context: [The situation and the forces at play]
Decision: [The choice that was made]
Consequences: [Positive consequences | Negative consequences]
Alternatives considered: [Other options and why they were not chosen]
```

Assessment — in the session, pass/fail

There is no Brightspace submission for this lab. During the session a demonstrator ticks three criteria at your machine:

1. The harness is green: your producer accepts a request on POST /process and the result is retrievable via GET /result/{id}.
2. At-least-once delivery is demonstrated live: the consumer acknowledges only after processing, and you can show a redelivery.
3. The dead-letter queue is bound correctly and a poisoned message lands in it rather than looping on the main queue.

Three ticks and this lab is passed. You work in pairs, and both of you must be able to explain any part of it on request.

The Labs Portfolio is 10% of the module and is pass/fail. You pass it by being ticked on at least three of the four labs and by submitting one ADR.

The one ADR — due Week 11

Across the whole trimester you submit a single Architectural Decision Record of at most 600 words, on whichever of the four labs you choose, by Friday of Week 11 on Brightspace. Use the template in Appendix A. It is judged competent or not yet competent against Appendix C of the module text; generic, uncontextualised reasoning is not competent.

Keep your code and evidence

Nothing is uploaded for this lab, but keep your repository, your README and any screenshots. Lab 1 material is in scope for the supervised lab exam in Week 10 — 90 minutes, individual, no AI assistants, drawn from Labs 1-3.